

Отчёт по лабораторной работе 8

Супонина Анастасия Павловна

Содержание

Цель работы	2
Задание	2
Выполнение лабораторной работы	2
Пакет Tikz для создания графиков	2
Введение.....	2
Первые линии	2
to и controls для построения кривых линий	3
Подписи.....	3
Подписи в фигурах	4
Объединение полученных знаний.....	4
Графики функции	5
Круг с разным радиусом	5
Треугольник Серпинского	6
Упражнение 1.....	7
Упражнение 2.....	7
Ковер Серпинского (Квадрат Серпинского).....	8
Список литературы.....	9

Список иллюстраций

package.....	2
how draw line	3
to and controls.....	3
make node.....	4
make a node with figure	4
example.....	5
function on graphic	5
circle	6
serpinsky triangle	6
exercise2	8

Цель работы

Изучить способы создания графиков при помощи Tikz в LaTeX.

Задание

Изучить все возможные способы создания графиков.

Выполнение лабораторной работы

Пакет Tikz для создания графиков

Введение

Для начала работы необходимо в среде LaTeX, загрузить нужный пакет как на изображении ниже



```
\usepackage{tikz}
```

package

Для создания изображения внутри документа, используем `begin` и `end` с аргументом `figure`, о котором мы говорили в прошлых лабораторных работах. Внутри же, чтобы прописывать код, который в дальнейшем станет изображением прописываем изначально `begin` и `end` с аргументом `tikzpicture`.

Первые линии

Для того, чтобы начать рисовать какие либо фигуры или линии в начале мы записываем команду `draw`, после нее мы вводим координаты в любом удобном формате в круглых скобках (через запятую, градусами и т.д). Также любой линии можно задать вид и то, как она будет идти. На рисунке ниже есть два графика, на первом указаны три точки и просто объединены обычной линией по умолчанию, на втором для первой линии мы задаем условие, чтобы она сначала шла горизонтально, а потом вертикально, объединяя наши точки под прямым углом и при этом в параметре `draw` записываем, что это должна быть стрелка, а во второй линии в качестве параметра задаем её цвет.

```

\begin{figure}
\begin{tikzpicture}
\draw (-1,0) -- (3,10pt) -- (35:3);
\end{tikzpicture}
\end{figure}

```



```

\begin{figure}
\begin{tikzpicture}
\draw[->] (-1,0) -| (3,10pt);
\draw[red] (3,10pt) -- (35:3);
\end{tikzpicture}
\end{figure}

```



how draw line

to и controls для построения кривых линий

Ниже на рисунки представлены для компонента Tikz: to и controls.

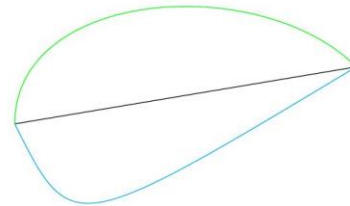
to - используется для того, чтобы задавать под каким углом линия выходит из первой точки (out) и под каким углом попадает во вторую (in)

controls - используется для того, чтобы задать точку или точки к которым линия будет изгибаться преодолевая расстояния между указанными точками

```

\begin{figure}
\begin{tikzpicture}
\draw (-1,0) to (5,1);
\draw[green] (-1,0) to[out=90,in=135] (5,1);
\draw[cyan] (-1,0) .. controls (0,-2) .. (5,1);
\end{tikzpicture}
\end{figure}

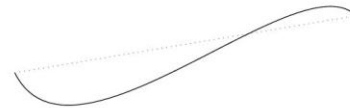
```



```

\begin{figure}
\begin{tikzpicture}
\draw[dotted,gray] (-1,0) -- (5,1);
\draw (-1,0) .. controls (0,-2) and (4,2) .. (5,1);
\end{tikzpicture}
\end{figure}

```



to and controls

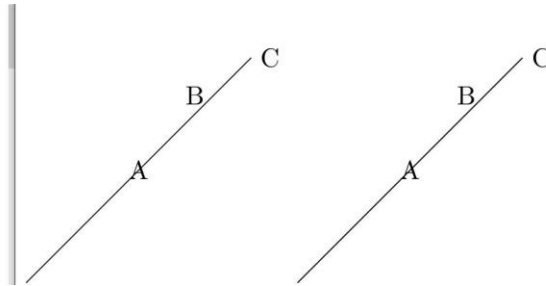
Подписи

Часто при создании графиков, нам нужно указывать названия точек или самих графиков или фигур, для этого в Tikz используется node, в квадратных скобках мы задаем все нужные параметры для наших обозначений, а в фигурных скобках пишем сами обозначения. Рисунок ниже иллюстрирует разницу в расположении node при использовании to.

```

\begin{figure}
\begin{tikzpicture}[scale=3]
\draw (0,0) -- (1,1) node[midway]{A}
node[pos=0.75,above]{B} node[right]{C};
\end{tikzpicture}
\begin{tikzpicture}[scale=3]
\draw (0,0) to node[midway]{A} node[pos=0.75,
above]{B} (1,1) node[right]{C};
\end{tikzpicture}
\end{figure}

```



make node

Подписи в фигурах

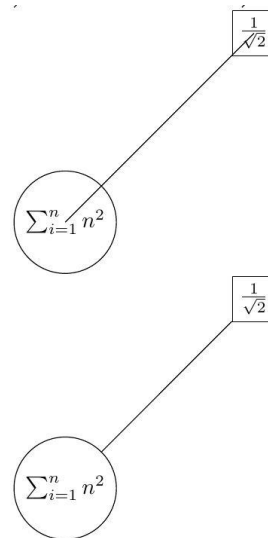
Иногда наши подписи нам нужно как-то выделить, мы уже обсудили ранее, что можем задавать разные параметры node, в них мы можем прописывать в какой фигуре будет лежать наша подпись. Однако, чтобы это выглядело хорошо, необходимо задавать эти точки со значениями отдельно. В круглых скобках к ним мы записываем их краткое обозначение по которому можно к ним обращаться для построения графика, и тогда линия график не будет их перекрывать. Все это наглядно отображено на рисунке ниже:

```

\begin{tikzpicture}[scale=3]
\draw (0,0) node[circle, draw]{ $\sum_{i=1}^n n^2$ }
-- (1,1)
node[rectangle, draw]{ $\frac{1}{\sqrt{2}}$ };
\end{tikzpicture}

\begin{tikzpicture}[scale=3]
% define nodes
\node[circle, draw] (label1) at (0,0) { $\sum_{i=1}^n n^2$ };
\node[rectangle, draw] (label2) at (1,1) { $\frac{1}{\sqrt{2}}$ };
% draw the line
\draw (label1) -- (label2);
\end{tikzpicture}
\end{figure}

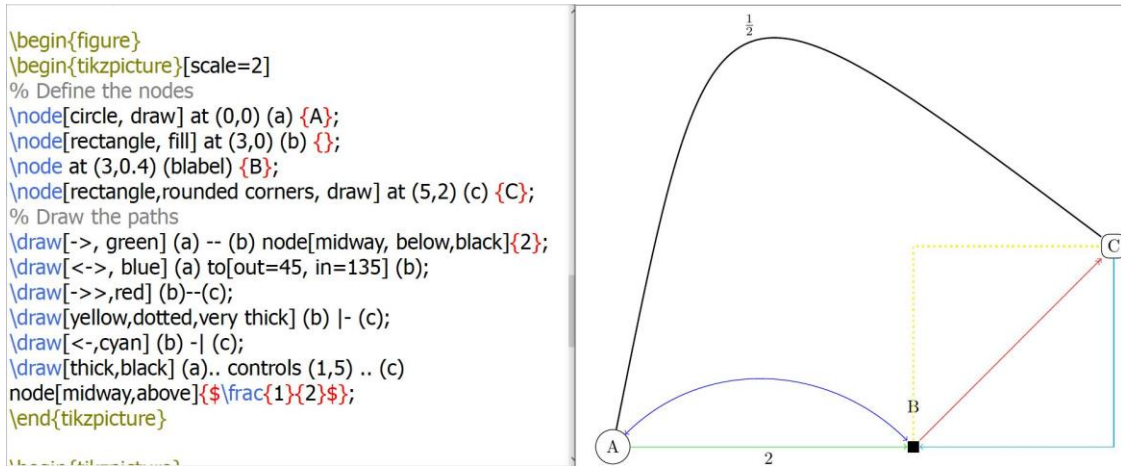
```



make a node with figure

Объединение полученных знаний

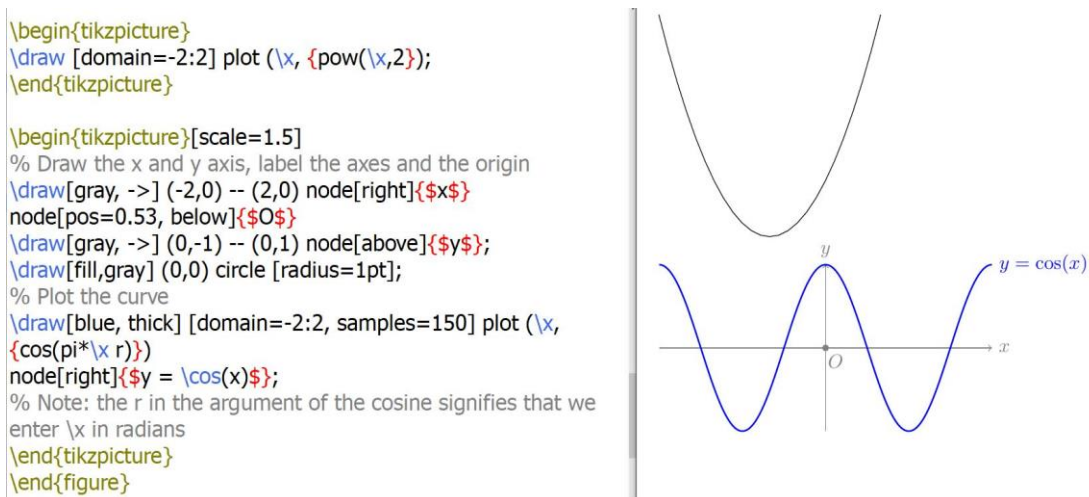
Ниже представлена иллюстрация включающая в себя все функции о которых мы говорили ранее



example

Графики функции

Мы говорили о произвольных линиях, которые мы можем строить, но что насчет графиков функций? Их мы тоже можем строить и для этого нужно использовать `plot`, однако не забываем перед ним указывать отрезок по x на котором будет строиться наша функция. Пример правильно оформления есть на изображении ниже, также там показан пример создания графика с осями координат, их создание происходит, также как и создание любых других стрелок, которое уже было нами обсуждено, но также можно рассмотреть этот пример, для проверки себя.



function on graphic

Круг с разным радиусом

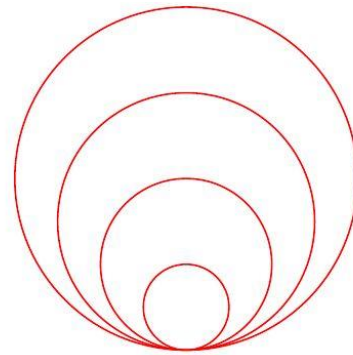
Ранее мы обсудили построение функций, где менялся x . также мы можем строить несколько фигур разного размера, на примере ни же это круг, для того, чтобы задать само изменения значения x (по сути как цикл `for` в питоне) используем `foreach`

```

\begin{figure}
\begin{tikzpicture}[scale=0.75]
\foreach \x in {0,1,2,3}
\draw[red,thick] (0,\x) circle [radius=\x+1];
\end{tikzpicture}
\end{figure}

\end{document}

```



circle

Треугольник Серпинского

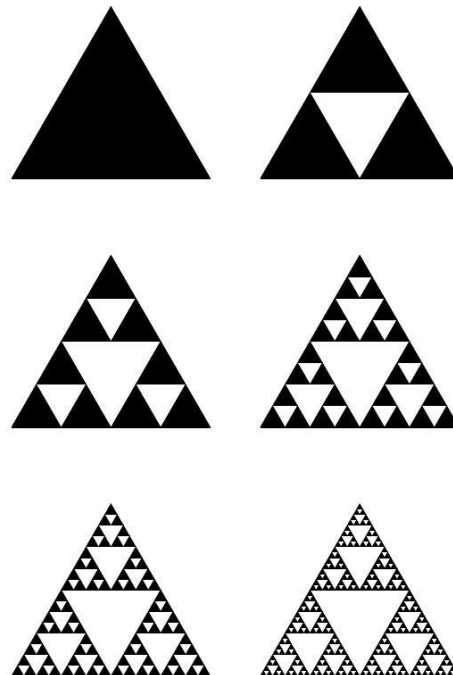
Треугольник серпинского это фрактальная фигура и её мы также можем задать в нашем Tikz.

Программа на изображении ниже генерирует последовательность фрактальных фигур Серпинского, используя рекурсивный алгоритм на языке TikZ. Рекурсивная функция `sierpinski` определяет построение равносторонних треугольников заданного масштаба и глубины. При глубине рекурсии, равной нулю, рисуется базовый треугольник; в противном случае функция делит исходный треугольник на три подобных и вызывает себя для каждого из них с уменьшенной глубиной. Цикл `for` последовательно строит шесть итераций фрактала от нулевого до пятого порядка, располагая их в виде матрицы 2×3 . Визуализация выполняется в среде `standalone` с единым размером границы.

```

\documentclass[border=1cm]{standalone}
\usepackage{tikz}
\usetikzlibrary{math}
% Define an equilateral triangle with lower left corner at coordinate #1 and
% with length of the sides #2
\newcommand\Triangle[2]{
  \draw #1 coordinate(a) -- ++(0:#2) coordinate(b);
  \draw (a) -- ++(60:#2) coordinate(c);
  \fill (a) -- (b) -- (c) -- cycle;
}
\begin{document}
\begin{tikzpicture}
\tikzmath{
  % Define the recursive function sierpinski
  function sierpinski(\x, \y, \s, \d) {
    if (\d == 0) then {
      % Draw a triangle lower left corner at (\x, \y), length \s
      \Triangle((\x,\y)){\s};
    } else {
      % Rescale the length of the sides and choose correct coords
      % for the next triangles
      \u1 = 0.25*\s;
      \u2 = \u1*sqrt(3);
      \u3 = 0.5*\s;
      sierpinski(\x,\y,\u3,\d-1);
      sierpinski(\x+\u3,\y,\u3,\d-1);
      sierpinski(\x+\u1,\y+\u2,\u3,\d-1);
    };
  };
  % Let the length of the sides of the base triangle be 4, and generate 6 figures
  \s = 4;
  for \d in {0,...,5}{
    % To situate all plots nicely under and next to each other, define the coords
    % of the lower left corners preemptively
    \x = (\s+1)*mod(\d,2);
    \y = int(\d/2) * (\s+1);
    sierpinski(\x,\y,\s,\d);
  };
}
\end{tikzpicture}
\end{document}

```

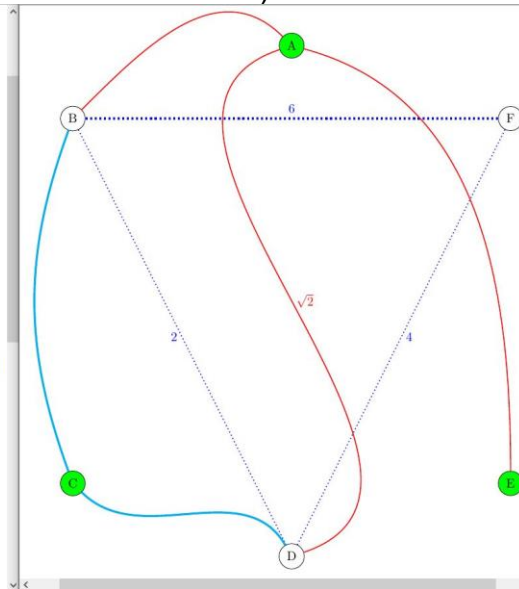


sierpinsky triangle

Упражнение 1

Был дан рисунок и было необходимо его повторить, ниже представлен код и результат. Для создания данного графика использовались `to` и `controls`, параметры подбирались методом перебора (или так сказать на глаз)

```
\begin{tikzpicture}[scale=2]
% Define the nodes
\node[circle, draw, fill=green] at (0,3) (a) {A};
\node[circle, draw, fill=green] at (-3,-3) (c) {C};
\node[circle, draw, fill=green] at (3,-3) (e) {E};
\node[circle, draw] at (3,2) (f) {F};
\node[circle, draw] at (-3,2) (b) {B};
\node[circle, draw] at (0,-4) (d) {D};
% Draw the paths
\draw[blue,dotted,ultra thick] (b) -- (f) node[midway, above]
{6};
\draw[blue,dotted,thick] (b) -- (d) node[midway, above, left]
{2};
\draw[blue,dotted,thick] (d) -- (f) node[midway, above, right]
{4};
\draw[red,thick] (d) .. controls (3,-3) and (-3,2) .. (a)
node[midway, above, right]{ $\sqrt{2}$ };
\draw[red,thick] (a) to[out=-15, in=90] (e);
\draw[red,thick] (b) .. controls (-2,3) and (-1,4) .. (a);
\draw[cyan,ultra thick] (b) to[out=-110, in=110] (c);
\draw[cyan,ultra thick] (c) to[out=-45, in=120] (d);
\end{tikzpicture}
```



Упражнение 2

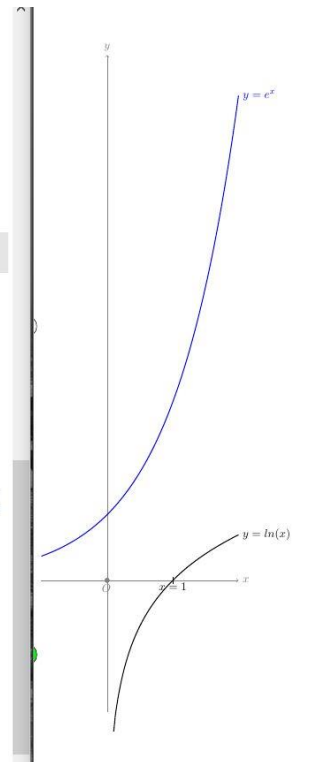
В данном упражнении также был график, который было необходимо повторить. Способ создания таких графиков был уже описан выше из интересного хотелось бы обратить внимание на параметр `samples` - это то, сколько точек используется при построении.

```

\begin{tikzpicture}[scale=2]
% Draw the x and y axis, label the axes and the
origin
\draw[gray, ->] (-1,0) -- (2,0) node[right]{$x$}
node[pos=0.33, below]{$O$};
\draw[gray, ->] (0,-2) -- (0,8) node[above]{$y$};
\draw[fill,gray] (0,0) circle [radius=1pt];
\draw (1,-0.05) -- (1,0.05) node[below, yshift=-2pt]
{$x=1$};

% Plot the curve
\draw[blue, thick] [domain=-1:2, samples=150] plot
(\x, {\exp(\x)}) node[right]{$y = e^x$};
\draw[black, thick] [domain=0.1:2, samples=150]
plot (\x, {\ln(\x)}) node[right]{$y = \ln(x)$};
% Note: the r in the argument of the cosine signifies
that we enter \x in radians
\end{tikzpicture}

```



exercise2

Ковер Серпинского (Квадрат Серпинского)

Используя рекурсивный подход, аналогичный ранее рассмотренному для треугольника Серпинского, данная программа строит квадратную фрактальную структуру — ковер Серпинского. В отличие от треугольного случая, где на каждом шаге рекурсивно обрабатывались три подобласти, здесь базовая геометрическая фигура — квадрат. При ненулевой глубине рекурсии сторона квадрата делится на три части, после чего функция `square` вызывается для восьми образовавшихся меньших квадратов, пропуская центральный. Логика пространственного расположения итераций с помощью цикла `for` и расчёта координат `(, )` остаётся идентичной предыдущему примеру, обеспечивая компактное расположение фракталов разных порядков в сетке.

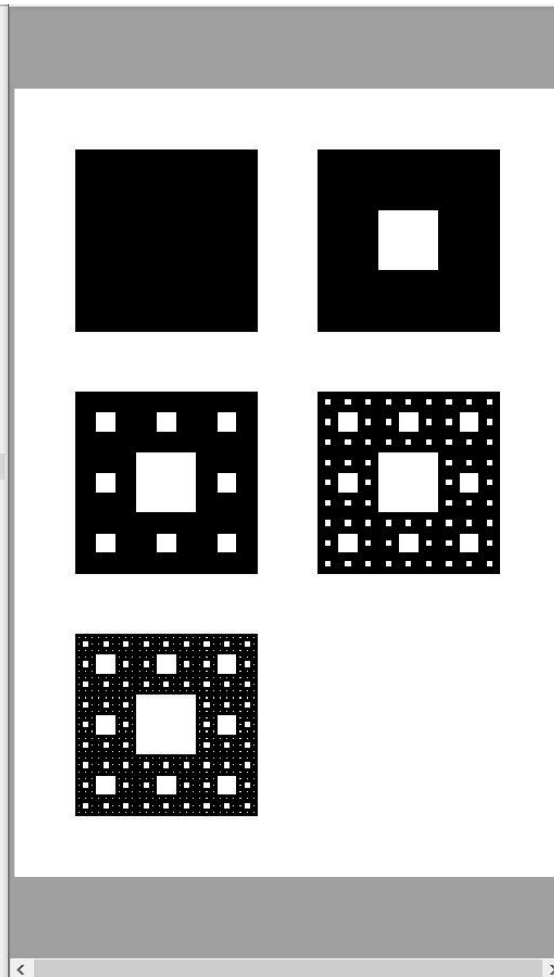

```

\documentclass[border=1cm]{standalone}
\usepackage{tikz}
\usetikzlibrary{math}

% Команда для квадрата
\newcommand\Square[2]{%
  \draw #1 rectangle ++(#2,#2);%
  \fill #1 rectangle ++(#2,#2);%
}

\begin{document}
\begin{tikzpicture}
\tikzmath{
  function carpet(\x,\y,\s,\d) {
    if (\d == 0) then {
      {{ \Square{(\x,\y)}{\s}; }};
    } else {
      \news = \s/3;
      carpet(\x, \y+2*\news, \news, \d-1);
      carpet(\x+\news, \y+2*\news, \news, \d-1);
      carpet(\x+2*\news, \y+2*\news, \news, \d-1);
      carpet(\x, \y+\news, \news, \d-1);
      carpet(\x+2*\news, \y+\news, \news, \d-1);
      carpet(\x, \y, \news, \d-1);
      carpet(\x+\news, \y, \news, \d-1);
      carpet(\x+2*\news, \y, \news, \d-1);
    };
  };
  \s = 3;
  for \d in {0,...,4} {
    \x = (\s+1)*mod(\d,2);
    \y = int(\d/2)*(\s+1);
    carpet(\x,\y,\s,\d);
  };
}
\end{tikzpicture}
\end{document}

```



В процессе выполнения данной лабораторной работы я научилась создавать графики разного типа в среде LaTeX.

Список литературы

::: Пособие по лабораторным работам [posobie]